Lecture 24 — Fenwick Trees & Sparse Tables

> Year Curriculum · Prerequisites: Week 7 (prefix sums, difference arrays), Week 9 (bit manipulation), Week 23 (segment trees) · Estimated time: 2 hours

1. Overview

The segment tree from Week 23 handles almost every range-query problem, but two special cases appear so often that they deserve their own, simpler tools:

- **Prefix sums with point updates.** The **Fenwick tree** (Binary Indexed Tree, BIT) stores exactly n numbers, is about ten lines of code, and runs several times faster than a segment tree. With small tricks it also handles range-add/point-query, range-add/range-sum, inversion counting, and "find the k-th element".
- **Static range minimum (or max, gcd, …).** The **sparse table** precomputes answers on all power-of-two windows in O(n log n) and then answers each query in **O(1)**.

Knowing which of the three structures to reach for — and implementing each without bugs in a few minutes — is a core contest skill.

2. Fenwick tree

2.1 Layout

Work 1-indexed. Let `lowbit(i) = i & -i`, the value of the lowest set bit of `i` (in two's complement, `-i = ~i + 1`, which flips every bit above the lowest one). Cell `t[i]` stores

```
t[i] = a[i - lowbit(i) + 1] + ... + a[i]
```

that is, the sum over the half-open range `(i − lowbit(i), i]`. For n = 8:

i	1	2	3	4	5	6	7	8
lowbit(i)	1	2	1	4	1	2	1	8
covers	[1,1]	[1,2]	[3,3]	[1,4]	[5,5]	[5,6]	[7,7]	[1,8]

2.2 Prefix query

To compute `prefix(i) = a[1] + … + a[i]`, add `t[i]` (which covers `(i − lowbit(i), i]`) and continue with `i − lowbit(i)`. Each step removes the lowest set bit, so there are at most ⌊log2 i⌋ + 1 steps. For i = 13 = 1101_2 we visit 13 (covering 13), 12 = 1100_2 (covering 9..12) and 8 = 1000_2 (covering 1..8): three disjoint blocks that tile `[1, 13]`.

2.3 Point update

To add `d` to `a[i]`, we must update every cell whose range contains `i`. These are exactly `i, i + lowbit(i), …` while `≤ n`: adding the lowest set bit moves to the next larger block that contains the current one. For i = 5 we visit 5, 6, 8 (and 16, 32, … if they exist). Again O(log n).

Range sums need subtraction: `sum(l, r) = prefix(r) − prefix(l − 1)`. This is the key limitation: a Fenwick tree answers range queries only for **invertible** operations (sum, xor, sums modulo p). It cannot answer arbitrary range minimum queries, because a minimum cannot be "subtracted".

2.4 Linear-time build

Calling `add` for each element costs O(n log n), which is almost always acceptable. An O(n) build: for i = 1..n, add `a[i]` into `t[i]`, then push `t[i]` into its parent `j = i + lowbit(i)` if `j ≤ n`.

3. Fenwick tree applications

3.1 Inversion counting

An inversion is a pair `i < j` with `a[i] > a[j]`. Compress the values to ranks `1..m` (sort, unique, `lower_bound`). Scan left to right with a BIT counting how many earlier elements have each rank. Before inserting `a[j]` with rank `r`, the number of earlier elements greater than it is `j − prefix(r)`. Total O(n log n). For `2 4 1 3 5`: when we see 1 there are 2 larger earlier elements (2 and 4), when we see 3 there is one (4) → 3 inversions. Inversion count equals the number of adjacent swaps bubble sort performs, which is why it appears in many disguises. The answer can reach about $n^2/2$, so use `long long`.

3.2 Range add, point query

Keep a BIT over the difference array `d[i] = a[i] − a[i−1]`. Adding `x` on `[l, r]` changes only `d[l] += x` and `d[r+1] −= x`, and `a[i] = d[1] + … + d[i] = prefix(i)`. This is exactly Week 7's difference array, but now updates and queries can interleave (CSES 1651 *Range Update Queries*; path-to-root sums on an Euler tour, Week 22).

3.3 Range add, range sum

We want `S(i) = a[1] + … + a[i]` with range-add updates. Since `a[k] = Σ_{j≤k} d[j]`,

```
S(i) = Σ_{k≤i} Σ_{j≤k} d[j] = Σ_{j≤i} d[j]·(i − j + 1) = i·Σ_{j≤i} d[j] − Σ_{j≤i} d[j]·(j − 1).
```

Keep one BIT `B1` over `d[j]` and another `B2` over `d[j]·(j−1)`. A range add of `x` on `[l, r]` changes `d[l]` by `+x` and `d[r+1]` by `−x`, so `B1` gets `+x` at `l` and `−x` at `r+1`, and `B2` gets `+x·(l−1)` at `l` and `−x·r` at `r+1`. Then `S(i) = i·B1.prefix(i) − B2.prefix(i)`. This is a lightweight replacement for the lazy segment tree of Week 25 when the only operations are "add on range" and "sum on range".

3.4 Finding the k-th element in O(log n)

If all values are non-negative (e.g., a 0/1 "present" array), the smallest `p` with `prefix(p) ≥ k` can be found by binary lifting on the BIT itself. Maintain `pos` with `prefix(pos) < k`, and try steps `2^LOG, …, 2, 1`: if `pos + step ≤ n` and `t[pos + step] < k`, then — because `pos` is a multiple of `2·step` at this moment, `t[pos+step]` covers exactly `(pos, pos+step]` — we can move to `pos + step` and subtract `t[pos + step]` from `k`. The answer is `pos + 1`. This is faster and shorter than binary search over `prefix` (which would be O(log² n)). CSES 1749 *List Removals* and Codeforces 1354D *Multiset* are direct applications.

3.5 Two-dimensional BIT

Nesting the loops gives a 2D Fenwick tree with O(log n · log m) update and prefix-rectangle query. Rectangle sums follow by inclusion–exclusion: `S(x2,y2) − S(x1−1,y2) − S(x2,y1−1) + S(x1−1,y1−1)`. Memory is n·m, so it fits grids up to about 1000×1000 (CSES 1739 *Forest Queries II*).

4. Sparse table

4.1 Construction

When the array never changes, we can afford to precompute more. Let `sp[k][i]` be the minimum of the window `a[i .. i + 2^k − 1]`. Windows of length 1 are the array itself, and a window of length 2^k is two windows of length 2^(k−1):

```
sp[0][i] = a[i]
sp[k][i] = min(sp[k−1][i], sp[k−1][i + 2^(k−1)])
```

There are ⌊log2 n⌋ + 1 levels with at most n entries each: O(n log n) time and memory.

4.2 O(1) queries for idempotent operations

For a query `[l, r]` let `len = r − l + 1` and `k = ⌊log2 len⌋`. The two windows `[l, l + 2^k − 1]` and `[r − 2^k + 1, r]` both lie inside `[l, r]` and together cover it (since `2·2^k > len`). They usually overlap, but for minimum that does not matter: `min(x, x) = x`. Operations with this property are called **idempotent**: min, max, gcd, bitwise AND and OR. So

```
min(a[l..r]) = min(sp[k][l], sp[k][r − 2^k + 1])
```

Precompute `lg[len]` with `lg[1] = 0`, `lg[i] = lg[i/2] + 1`. Avoid floating-point `log2`, which can be off by one at exact powers of two; the GCC builtin `__lg(x)` (index of the highest set bit) is a safe alternative.

For non-idempotent operations (sum, xor), decompose `[l, r]` into **disjoint** power-of-two windows by walking the bits of the length, which takes O(log n). For static sums, though, ordinary prefix sums are simply better.

4.3 Applications

- LCA in O(1): range-minimum of depths on the full Euler tour (Week 22).
- Binary search or two pointers with O(1) range-min/max/gcd checks: e.g., Codeforces 689D *Friends and Subsequences* counts intervals where `max(a[l..r]) = min(b[l..r])` by binary searching with two sparse tables; Codeforces 1549D *Integers Have Friends* uses a gcd sparse table with two pointers.
- LCP queries on a suffix array (Strings II and the Nationals track).

5. Choosing between them

	Fenwick tree	Sparse table	Segment tree
Updates	point (range via tricks)	none	point; range with lazy
Queries	invertible ops (sum, xor)	idempotent ops (min, max, gcd)	any associative op
Query time	O(log n)	O(1)	O(log n)
Memory	n	n log n	2n–4n
Code size	~10 lines	~15 lines	20–40 lines

Rules of thumb: dynamic sums → Fenwick; static min/max/gcd with many queries → sparse table; anything else → segment tree.

6. Reference implementations

6.1 Inversion count with coordinate compression

```cpp
#include <bits/stdc++.h>
using namespace std;

// Count inversions (pairs i<j with a[i] > a[j]) with a Fenwick tree
// over compressed values. O(n log n).
struct Fenwick {
    int n;
    vector<long long> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, long long d) { for (; i <= n; i += i & -i) t[i] += d; }
    long long prefix(int i) const { long long s = 0; for (; i > 0; i -= i & -i) s +=
t[i]; return s; }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto& x : a) cin >> x;
    vector<long long> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    Fenwick fw(vals.size());
    long long inv = 0;
    for (int j = 0; j < n; j++) {
        int r = lower_bound(vals.begin(), vals.end(), a[j]) - vals.begin() + 1; // 1-
based rank
        inv += j - fw.prefix(r);     // earlier elements with rank > r
        fw.add(r, 1);
    }
    cout << inv << '\n';
}
```

6.2 Fenwick k-th element (CSES 1749 *List Removals*)

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1749 - List Removals: Fenwick tree of 0/1 "still present" flags,
// with an O(log n) descent to find the k-th present element.
struct Fenwick {
    int n, LOG;
    vector<int> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {
```

```
        LOG = 1;
        while ((1 << LOG) <= n) LOG++;
    }
    void add(int i, int d) { for (; i <= n; i += i & -i) t[i] += d; }
    // smallest index p with prefix(p) >= k (assumes 1 <= k <= total)
    int kth(int k) const {
        int pos = 0;
        for (int step = 1 << (LOG - 1); step > 0; step >>= 1) {
            if (pos + step <= n && t[pos + step] < k) {
                pos += step;
                k -= t[pos];
            }
        }
        return pos + 1;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> x(n + 1);
    for (int i = 1; i <= n; i++) cin >> x[i];
    Fenwick fw(n);
    for (int i = 1; i <= n; i++) fw.add(i, 1);
    for (int i = 0; i < n; i++) {
        int p;
        cin >> p;
        int idx = fw.kth(p);
        cout << x[idx] << ' ';
        fw.add(idx, -1);
    }
    cout << '\n';
}
```

6.3 Range add + range sum with two BITs (with a self-test)

This program checks the two-BIT structure against a brute force on 2000 random test cases; the `RangeBIT` struct can be copied as is.

```
#include <bits/stdc++.h>
using namespace std;

// Range add + range sum with two Fenwick trees (tested against brute force).
struct RangeBIT {
    int n;
    vector<long long> b1, b2;
    RangeBIT(int n) : n(n), b1(n + 2, 0), b2(n + 2, 0) {}
    void upd(vector<long long>& b, int i, long long d) { for (; i <= n; i += i & -i) b[i]
+= d; }
    long long qry(const vector<long long>& b, int i) const { long long s = 0; for (; i >
```

```
0; i -= i & -i) s += b[i]; return s; }
    void rangeAdd(int l, int r, long long x) {   // a[l..r] += x, 1-indexed
        upd(b1, l, x);      upd(b1, r + 1, -x);
        upd(b2, l, x * (l - 1)); upd(b2, r + 1, -x * r);
    }
    long long prefix(int i) const { return qry(b1, i) * i - qry(b2, i); }
    long long rangeSum(int l, int r) const { return prefix(r) - prefix(l - 1); }
};

int main() {
    mt19937 rng(7);
    for (int it = 0; it < 2000; it++) {
        int n = rng() % 15 + 1;
        RangeBIT bit(n);
        vector<long long> a(n + 1, 0);
        for (int op = 0; op < 30; op++) {
            int l = rng() % n + 1, r = rng() % n + 1;
            if (l > r) swap(l, r);
            if (rng() % 2) {
                long long x = (long long)(rng() % 21) - 10;
                bit.rangeAdd(l, r, x);
                for (int i = l; i <= r; i++) a[i] += x;
            } else {
                long long s = 0;
                for (int i = l; i <= r; i++) s += a[i];
                if (s != bit.rangeSum(l, r)) { puts("MISMATCH"); return 1; }
            }
        }
    }
    puts("all tests passed");
}
```

6.4 Sparse table (CSES 1647 *Static Range Minimum Queries*)

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1647 - Static Range Minimum Queries: sparse table, O(1) per query.
struct SparseTable {
    vector<vector<int>> sp;     // sp[k][i] = min of a[i .. i + 2^k - 1]
    vector<int> lg;             // lg[x] = floor(log2 x)
    SparseTable(const vector<int>& a) {
        int n = a.size();
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        int K = lg[n] + 1;
        sp.assign(K, vector<int>(n));
        sp[0] = a;
        for (int k = 1; k < K; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                sp[k][i] = min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
    }
    int query(int l, int r) const {          // inclusive, 0-indexed, l <= r
```

```
        int k = lg[r - l + 1];
        return min(sp[k][l], sp[k][r - (1 << k) + 1]);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int& x : a) cin >> x;
    SparseTable st(a);
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << st.query(l - 1, r - 1) << '\n';
    }
}
```

7. Common mistakes

- **Index 0 in a Fenwick tree.** `0 & -0 = 0`, so `add(0, x)` loops forever. Shift to 1-indexed.
- **`r + 1` out of range** in range-add. Either check `r + 1 ≤ n` inside `add` (the loop condition already does) or allocate `n + 2`.
- **Using a BIT for range minimum** with arbitrary updates. It does not work; use a segment tree.
- **k-th search with negative values.** The descent assumes prefix sums are non-decreasing.
- **Sparse table with floating-point `log2`.** Precompute `lg[]` or use `__lg`.
- **Sparse table memory.** `n = 10^6` with 20 levels of `int` is 80 MB; with `long long`, 160 MB. Check limits or use `vector<int>` per level sized `n - 2^k + 1`.
- **Overlapping windows for sums** double count.
- **Forgetting compression** when the BIT is indexed by values up to 10^9.

8. Practice problems

Warm-up:

- CSES 1648 — *Dynamic Range Sum Queries* — https://cses.fi/problemset/task/1648
- CSES 1647 — *Static Range Minimum Queries* — https://cses.fi/problemset/task/1647
- CSES 1651 — *Range Update Queries* — https://cses.fi/problemset/task/1651
- SPOJ INVCNT — *Inversion Count* — https://www.spoj.com/problems/INVCNT/

Core:

- CSES 1749 — *List Removals* — https://cses.fi/problemset/task/1749
- CSES 1739 — *Forest Queries II* — https://cses.fi/problemset/task/1739
- Codeforces 61E — *Enemy is weak* — https://codeforces.com/problemset/problem/61/E
- Codeforces 359D — *Pair of Numbers* — https://codeforces.com/problemset/problem/359/D

Challenge:

- Codeforces 1354D — *Multiset* — https://codeforces.com/problemset/problem/1354/D
- Codeforces 689D — *Friends and Subsequences* — https://codeforces.com/problemset/problem/689/D
- Codeforces 1549D — *Integers Have Friends* — https://codeforces.com/problemset/problem/1549/D
- CSES 1734 — *Distinct Values Queries* (offline + BIT) — https://cses.fi/problemset/task/1734

9. Further reading

- CP-Algorithms, "Fenwick Tree" — https://cp-algorithms.com/data_structures/fenwick.html
- CP-Algorithms, "Sparse Table" — https://cp-algorithms.com/data_structures/sparse-table.html
- Peter M. Fenwick, "A New Data Structure for Cumulative Frequency Tables", *Software: Practice and Experience*, 1994.
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 9.
- Next: Week 25 extends segment trees with lazy propagation for general range updates.